

ATIVIDADE REMOTA — MÁQUINAS DE TURING

Teoria da Computação

Estudo, simulação e reflexão sobre os limites computacionais

Disciplina

Teoria da Computação

Tema

Máquinas de Turing

Modalidade

Remota

Valor

1,0 ponto



Material obrigatório

Assista ao vídeo antes de realizar a atividade

Para realizar esta atividade, assista ao vídeo:

Akitando #86 — O Computador de Turing e Von Neumann: Por que calculadoras não são computadores?

Orientação: Assista ao material com atenção e utilize os conceitos apresentados para responder às questões e desenvolver a atividade de simulação.

1. Objetivos da atividade

Ao final da atividade, o estudante deverá ser capaz de:

- Compreender o conceito de Máquina de Turing.
- Identificar os principais componentes de uma Máquina de Turing.
- Compreender a importância das Máquinas de Turing para a computação.
- Construir e simular uma Máquina de Turing simples.
- Reconhecer que existem problemas que não podem ser resolvidos por algoritmos, compreendendo os limites da computação.

2. Conteúdo

- Conceito de Máquina de Turing.
- Fita, cabeça de leitura/escrita e estados.
- Alfabeto e regras de transição.
- Funcionamento de uma Máquina de Turing.
- Simulação computacional.
- Computabilidade e limites computacionais.

3. Desenvolvimento da atividade

Etapa 1 — Introdução

Após assistir ao vídeo e estudar o material disponibilizado, responda às questões abaixo.

1. O que é uma Máquina de Turing?

É um modelo matemático de computador criado por Alan Turing em 1936. Não é uma máquina física: é uma fita infinita dividida em células, uma cabeça que lê e escreve um símbolo por vez e anda para a esquerda ou para a direita, e uma tabela de regras que diz o que fazer em cada situação. Mesmo sendo simples, ela consegue executar qualquer algoritmo, por isso virou a definição do que é "computar".

2. Quais são os principais componentes de uma Máquina de Turing?

Fita: infinita, dividida em células; guarda a entrada, os rascunhos e a saída.
Cabeça de leitura/escrita: lê o símbolo da célula atual, escreve outro e anda uma casa (E ou D).
Estados: conjunto finito, com um estado inicial e os estados de parada (aceita/rejeita).
Alfabeto: os símbolos que podem aparecer na fita, incluindo o branco.
Transições: regras do tipo (estado, símbolo lido) -> (símbolo escrito, movimento, próximo estado).

3. Qual é a importância das Máquinas de Turing para a computação?

Ela deu uma definição exata do que é um algoritmo. Com ela dá para provar o que um computador consegue fazer e o que ele nunca vai conseguir, como o problema da parada. Ela também é a base do computador moderno: Von Neumann usou a ideia de máquina universal e colocou programa e dados na mesma memória. Toda linguagem de programação atual (Python, Java, C) é Turing completa, ou seja, faz o mesmo que uma Máquina de Turing.

4. Qual é a relação entre Máquina de Turing e algoritmo?

Um algoritmo é uma sequência finita de passos bem definidos, e a Máquina de Turing é a forma matemática de escrever isso: cada regra da tabela é um passo. Pela tese de Church-Turing, tudo que pode ser resolvido por um algoritmo pode ser resolvido por uma Máquina de Turing. Então, se não existe Máquina de Turing para um problema, não existe algoritmo para ele.

Etapa 2 — Simulação

Utilize um software ou simulador de Máquina de Turing indicado pelo professor.

Crie uma máquina capaz de reconhecer palavras da forma:

$$0^n 1^n$$

Exemplos aceitos

- 01
- 0011
- 000111
- 00001111

Exemplos rejeitados

- 0
- 1
- 001
- 011
- 00111

Desafio: A máquina deverá verificar se existe a mesma quantidade de símbolos **0** e **1**, seguindo a lógica de funcionamento de uma Máquina de Turing.

Etapa 3 — Registro da simulação

Após executar a máquina, registre:

- A palavra utilizada como entrada.
- Os estados percorridos.
- O resultado da execução: **ACEITA** ou **REJEITA**.
- Uma captura de tela da simulação.

Realize pelo menos **3 testes**:

- 2 entradas que devem ser aceitas.
- 1 entrada que deve ser rejeitada.

Registro dos testes

Teste	Entrada	Resultado esperado	Resultado obtido	Estados percorridos
1	0011	ACEITA	ACEITA	$q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_0 \rightarrow q_3 \rightarrow \text{qaceita}$ (13 passos)
2	000111	ACEITA	ACEITA	$q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_0 \rightarrow q_3 \rightarrow \text{qaceita}$ (25 passos)
3	00111	REJEITA	REJEITA	$q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_0 \rightarrow q_1 \rightarrow q_2 \rightarrow q_0 \rightarrow q_3$ (parou em q_3 lendo 1: não há regra)

Descrição da Máquina de Turing criada:

A máquina risca um 0 e um 1 a cada volta. Em q_0 ela troca o primeiro 0 por X e vai para q_1 , que anda para a direita pulando 0 e Y até o primeiro 1, que vira Y. Em q_2 ela volta para a esquerda até o X e recomeça em q_0 . Quando q_0 encontra Y em vez de 0, os 0 acabaram, e q_3 confere se até o branco só tem Y: se tiver, ACEITA. Se sobrar algum 0 ou 1 sem par, não existe regra e ela REJEITA. Estados: q_0 , q_1 , q_2 , q_3 , qaceita. Tabela de transições e capturas no anexo (páginas 6 a 8).

Etapa 4 — Reflexão sobre os limites computacionais

Uma Máquina de Turing consegue resolver qualquer problema? Explique com suas palavras por que existem problemas que não podem ser resolvidos por algoritmos.

Não. A Máquina de Turing consegue fazer tudo que um algoritmo consegue, mas existem problemas que nenhum algoritmo resolve. O exemplo clássico é o problema da parada: decidir, para qualquer programa e qualquer entrada, se ele vai terminar ou rodar para sempre. Turing provou que isso é impossível. A ideia da prova é que, se existisse esse "detector", daria para construir um programa que pergunta ao detector sobre si mesmo e faz o contrário da resposta, e o detector erraria de qualquer jeito. Então o limite não é falta de memória ou de velocidade: mesmo com fita infinita e tempo infinito, esses problemas continuam sem solução, porque o próprio conceito de algoritmo não alcança eles.

Anexo - Máquina de Turing para 0^n1^n

Aluno: João Gabriel. A simulação foi feita em um simulador que eu escrevi em Python (maquina_turing.py).
Ideia: a cada volta o 0 mais à esquerda vira X e o 1 mais à esquerda vira Y. Se no fim só sobrar X e Y, a quantidade de 0 e 1 era igual e a palavra é aceita.

Estados:

q0 começo da volta; procura o próximo 0
q1 já riscou um 0; anda para a direita até o primeiro 1
q2 já riscou o 1; volta para a esquerda até o X
q3 acabaram os 0; confere se só tem Y até o fim
qaceita parada com aceitação

Tabela de transições (_ = branco, D = direita, E = esquerda):

Estado	Lê	Escreve	Move	Vai para
q0	0	X	D	q1
q0	Y	Y	D	q3
q1	0	0	D	q1
q1	Y	Y	D	q1
q1	1	Y	E	q2
q2	0	0	E	q2
q2	Y	Y	E	q2
q2	X	X	D	q0
q3	Y	Y	D	q3
q3	—	—	D	qaceita

Qualquer situação que não está na tabela faz a máquina parar e REJEITAR
(ex.: q3 lendo 1 = sobrou 1 sem par; q1 lendo branco = sobrou 0 sem par).

Testes extras: 01 e 00001111 ACEITA; 0, 1, 001 e 011 REJEITA.
Todos bateram com o esperado.

Capturas de tela da simulação

Teste 1 - entrada 0011 (ACEITA)

```
Windows PowerShell

PS C:\estudos\ia\Unidade4_MaquinaDeTuring> python maquina_turing.py 0011

Entrada: 0011
passo estado  fita
 0 q0      [0] 0 1 1 _
 1 q1      X [0] 1 1 _
 2 q1      X 0 [1] 1 _
 3 q2      X [0] Y 1 _
 4 q2      [X] 0 Y 1 _
 5 q0      X [0] Y 1 _
 6 q1      X X [Y] 1 _
 7 q1      X X Y [1] _
 8 q2      X X [Y] Y _
 9 q2      X [X] Y Y _
10 q0      X X [Y] Y _
11 q3      X X Y [Y] _
12 q3      X X Y Y [_]
13 qaceita X X Y Y _

Resultado: ACEITA
Estados: q0 → q1 → q2 → q0 → q1 → q2 → q0 → q3 → qaceita

PS C:\estudos\ia\Unidade4_MaquinaDeTuring>
```

Teste 2 - entrada 000111 (ACEITA)

```
Windows PowerShell

PS C:\estudos\ia\Unidade4_MaquinaDeTuring> python maquina_turing.py 000111

Entrada: 000111
passo estado  fita
 0 q0      [0] 0 0 1 1 1 _
 1 q1      X [0] 0 1 1 1 _
 2 q1      X 0 [0] 1 1 1 _
 3 q1      X 0 0 [1] 1 1 _
 4 q2      X 0 [0] Y 1 1 _
 5 q2      X [0] 0 Y 1 1 _
 6 q2      [X] 0 0 Y 1 1 _
 7 q0      X [0] 0 Y 1 1 _
 8 q1      X X [0] Y 1 1 _
 9 q1      X X 0 [Y] 1 1 _
10 q1      X X 0 Y [1] 1 _
11 q2      X X 0 [Y] Y 1 _
12 q2      X X [0] Y Y 1 _
13 q2      X [X] 0 Y Y 1 _
14 q0      X X [0] Y Y 1 _
15 q1      X X X [Y] Y 1 _
16 q1      X X X Y [Y] 1 _
17 q1      X X X Y Y [1] _
18 q2      X X X Y [Y] Y _
19 q2      X X X [Y] Y Y _
20 q2      X X [X] Y Y Y _
21 q0      X X X [Y] Y Y _
22 q3      X X X Y [Y] Y _
23 q3      X X X Y Y [Y] _
24 q3      X X X Y Y Y [_]
25 qaceita X X X Y Y Y _

Resultado: ACEITA
Estados: q0 → q1 → q2 → q0 → q1 → q2 → q0 → q1 → q2 → q0 → q3 → qaceita

PS C:\estudos\ia\Unidade4_MaquinaDeTuring>
```

Teste 3 - entrada 00111 (REJEITA)

Windows PowerShell

```
PS C:\estudos\ia\Unidade4_MaquinaDeTuring> python maquina_turing.py 00111
```

Entrada: 00111

passo	estado	fita
0	q0	[0] 0 1 1 1 _
1	q1	X [0] 1 1 1 _
2	q1	X 0 [1] 1 1 _
3	q2	X [0] Y 1 1 _
4	q2	[X] 0 Y 1 1 _
5	q0	X [0] Y 1 1 _
6	q1	X X [Y] 1 1 _
7	q1	X X Y [1] 1 _
8	q2	X X [Y] Y 1 _
9	q2	X [X] Y Y 1 _
10	q0	X X [Y] Y 1 _
11	q3	X X Y [Y] 1 _
12	q3	X X Y Y [1] _

Resultado: REJEITA

Estados: q0 → q1 → q2 → q0 → q1 → q2 → q0 → q3

```
PS C:\estudos\ia\Unidade4_MaquinaDeTuring>
```

4. Entrega

O estudante deverá enviar **um único arquivo** contendo:

- Respostas da Etapa 1;
- Descrição da Máquina de Turing criada;
- Capturas de tela da simulação;
- Resultados dos 3 testes;
- Resposta da reflexão sobre os limites computacionais.

Formato: PDF ou documento do Word.

5. Critérios de avaliação — 1,0 ponto

Critério	Valor
Compreensão do conceito e importância das Máquinas de Turing	0,20
Construção/configuração da Máquina de Turing	0,30
Realização e registro da simulação	0,20
Interpretação dos resultados	0,10
Compreensão dos limites computacionais	0,20
Total	1,00

6. Questão final

Problema para reflexão:

Imagine que você recebeu um problema computacional muito complexo. Como saber se ele é apenas difícil de resolver ou se, na verdade, não existe nenhum algoritmo capaz de resolvê-lo para todos os casos?

Explique utilizando os conceitos estudados sobre **Máquinas de Turing, computabilidade e limites computacionais**.

Um problema difícil tem algoritmo, só que ele demora muito, como testar todas as combinações de uma senha: é lento, mas termina e dá a resposta certa. Isso é assunto de complexidade ($O(n^2)$, $O(2^n)$).

Um problema sem algoritmo (indecidível) é diferente: não existe Máquina de Turing que pare com a resposta certa para todas as entradas. Para saber se é o caso, usa-se a redução: mostrar que, se eu conseguisse resolver o meu problema, eu também resolveria o problema da parada. Como o da parada já foi provado impossível, o meu também é.

Na prática: se consigo montar uma máquina que sempre para com a resposta certa, é computável (mesmo que lento); se resolver ele resolveria o problema da parada, é indecidível. Só ver que o programa "está demorando" não prova nada: demora é sinal de problema difícil, não de problema impossível.



Orientações finais

1. Assista ao vídeo indicado.
2. Estude os conceitos apresentados.
3. Responda às questões propostas.
4. Realize a simulação da Máquina de Turing.
5. Faça os três testes solicitados.
6. Registre os resultados e as evidências da simulação.
7. Responda à reflexão sobre os limites computacionais.
8. Organize o material em um único arquivo.
9. Envie o arquivo conforme as orientações do professor.

Teoria da Computação — Atividade avaliativa | Profª Kadidja Valéria | Linguagens Formais e Autômatos |

Máquinas de Turing • Computabilidade • Limites Computacionais